Acceso a ficheros

PERSISTENCIA EN FICHEROS
CON JAVA

I/O Input/Output - Entrada/Salida de la información

Todo dato tiene un origen de entrada(**Input**) o un destino de salida (**Output)**

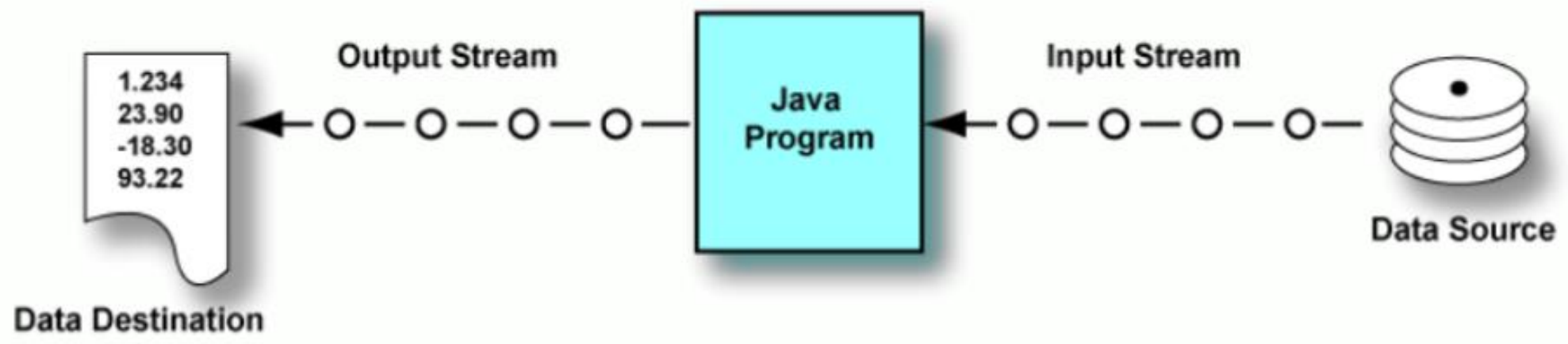
- **Input** implica leer datos de una fuente y
- **Output** implica escribirlos en un destino.

Un programa trabaja con datos que provienen de una entrada la cual puede ser un teclado, un archivo en disco o un socket a través de la red.

Un programa genera resultados enviándolos a una salida la cual puede ser la pantalla, un archivo en disco o un socket a través de la red.

En Java, la **conexión entre un programa y un origen o destino** de datos se denomina **flujo** (en inglés: **stream**).

I/O Input/Output - Entrada/Salida de la información



Un flujo de entrada *(**input stream**)* maneja los datos que fluyen hacia un programa. Un flujo de salida *(**output stream**)* maneja los datos que salen de un programa.

Interactuando con el usuario: E/S estándar

Java proporciona soporte para la **entrada/salida estándar** a través de tres streams:

- **System.in**: se correspondería con la **entrada estándar**, permitiría introducir datos por teclado a nuestro programa.
- **System.out**: se correspondería con la **salida estándar**, por lo general, se usa una pantalla de ordenador para mostrar los resultados del programa.
- **System.err**: se correspondería con el error estándar, y normalmente se redirige a System.out.

Interactuando con el usuario: E/S estándar

Para imprimir información en la terminal usaríamos algo parecido a esto:

```
System.out.println("mensaje simple");
System.err.println("mensaje de error");
```

Y para leer información del usuario, esto:

```
int i=System.in.read(); //Devuelve el código ASCII del primer carácter
System.out.println((char)i); //imprimirá el carácter, si no convertimos imprime el ASCII
```

Imprimir información en un terminal de texto es simple (**System.out.println**), pero para leer información puede ser un proceso difícil.

Java nos ofrece varias herramientas, una de ellas es la clase **Scanner**.

Tipos de Datos - Caracteres y bytes

Existen dos tipos de flujos (streams) según el tipo de datos:

- **flujos binarios** (byte streams): **leen/escriben bytes**, que **no** son **legibles para el ser humano** (variables, clases del programa, archivos de imagen, sonido, etc).

  Las clases principales para manejar estos flujos son las clases abstractas

  InputStream y OutputStream

- **flujos de caracteres** (character streams): **leen/escriben caracteres** , **legibles para el ser humano** (ficheros de texto, xml y json).

  Las clases principales para manejar estos flujos son las clases abstractas

  Reader y Writer

  Los métodos que se utilizan son similares, pero se utilizan clases diferentes.

Clases principales de **OutputStream**:

Clase	Función
`ByteArrayOutputStream`	Crea un buffer en la memoria, todos los datos transmitidos en el flujo se colocan en este buffer.
`FileOutputStream`	Para transmitir información de Archivo
`PipeOutputStream`	Implementa el concepto de tubería (pipe)
`BufferedOutputStream`	Añade la función de almacenamiento en buffer a las operaciones de escritura
`DataOutputStream`	Proporciona funcionalidad para escribir en archivos de tipo primitivo de java

Clases principales de **InputStream**:

Clase	Función
`ByteArrayInputStream`	Permite utilizar los buffers de memoria como InputStream
`StringBufferInputStream`	Convierte un string en InputStream
`FileInputStream`	Para leer la información del Archivo
`PipeInputStream`	Implementa el concepto de tubería (pipe)
`SequenceInputStream`	Convierte dos o más objetos InputStream en un único InputStream
`BufferedInputStream`	Añade la función de almacenamiento en buffer a las operaciones de lectura
`DataInputStream`	Proporciona una funcionalidad para leer de un flujo de tipos primitivos Java

Clases principales de **Writer**:

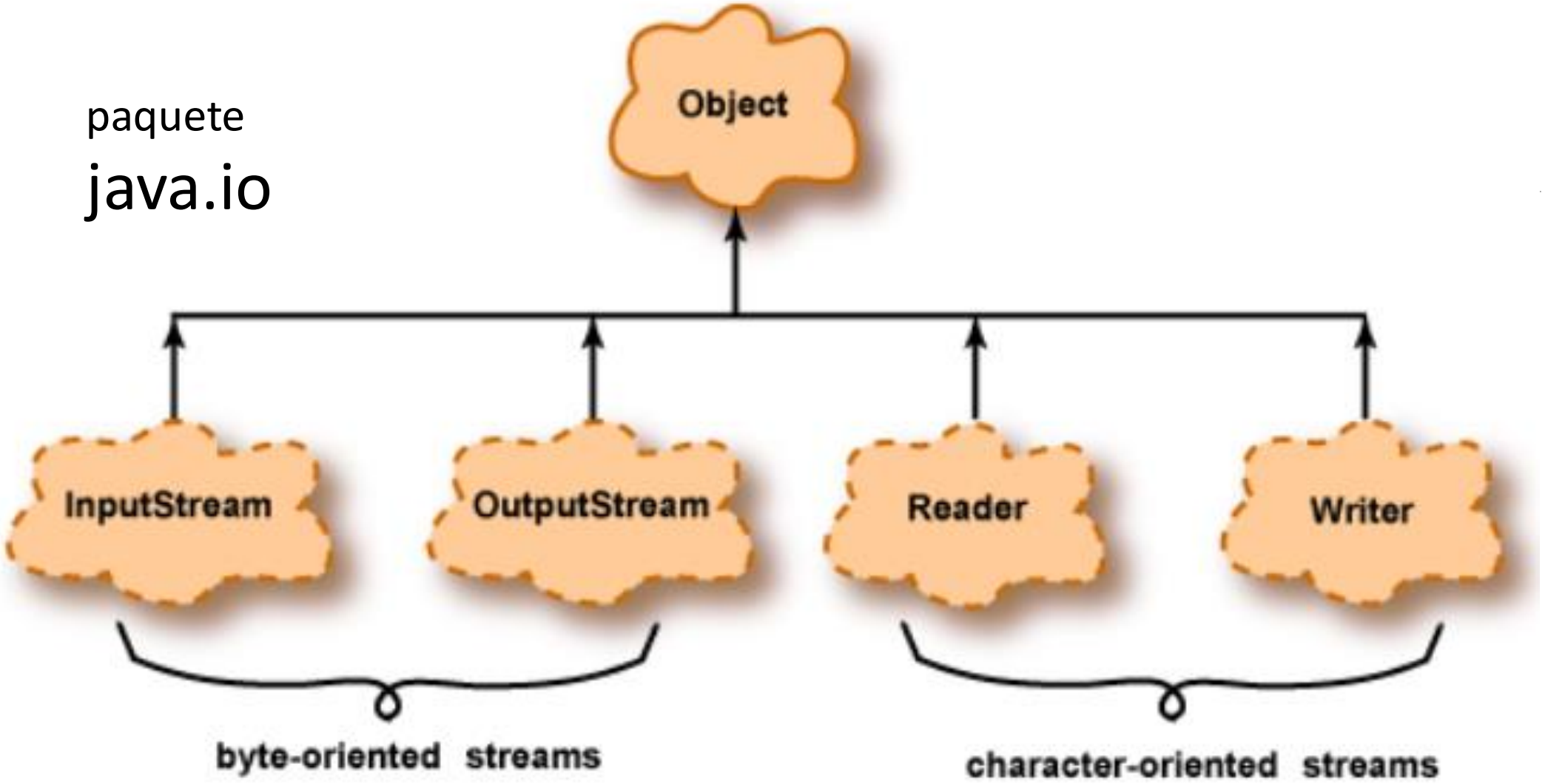
`StringWriter`	Permite escribir stream manipulados como cadenas
`OutputStreamWriter`	Es una clase puente entre el stream de char y el stream de bytes.
`FileWriter`	Proporciona soporte para la escritura de caracteres en un archivo
`PipedWriter`	Permite escribir caracteres en una pipe
`PrintWriter`	Ofrece soporte para la impresión de caracteres con formato
`BufferedWriter`	Añade la funcionalidad de almacenamiento en buffer al flujo para que no haya necesidad de acceder al sistema de archivos para cada operación de escritura.

Clases principales de **Reader**:

`StringReader`	Representa un stream que opera con cadenas
`InputStreamReader`	Es una clase puente entre el stream de char y el stream de bytes.
`FileReader`	Proporciona soporte para la lectura de caracteres desde un archivo
`PipedReader`	Permite leer los caracteres de una pipe
`BufferedReader`	Añade la función de almacenamiento en buffer al stream para que no sea necesario acceder al sistema de archivos para cada operación de lectura
`LineNumberReader`	Mantiene un registro del número de línea desde el que se leen los caracteres

paquete

java.io



Tipos de datos

- **Flujos orientados a bytes**
  - Diseñado para entrada y salida de propósito general.
  - Los datos pueden ser tipos de datos primitivos o bytes sin procesar.

- **Flujos orientados a caracteres**
  - Destinado a datos de caracteres (archivos de texto).
  - Los datos se transforman de/a caracteres Java de 16 bits utilizados dentro de los programas al formato utilizado externamente.

Ficheros

Un fichero es un **conjunto lógico de información o de datos** que se designa con un **nombre** y que se almacenan en el sistema de archivos del ordenador.

Un archivo o fichero informático es un **conjunto de bits que son almacenados en un dispositivo**.

Para poder **acceder a ellos** haremos uso de una **ruta (path)** ya sea **relativa o absoluta**.

```
../Prog2/Exemple/src/Main.java  → Ruta relativa

C:\User\guillermo\Prog2\Exemple\src\Main.java → Ruta absoluta
```

Tipos de acceso a ficheros

- **Acceso secuencial**: se accede a los datos en el orden en que están almacenados, desde el principio hasta el final, es decir, de una manera lineal (sin saltos).
  - **Lectura secuencial**: Se puede realizar mediante clases como **FileReader (para archivos de texto) o InputStream (para archivos binarios)**.
  - **Escritura secuencial**: Se puede escribir con clases **como FileWriter (para archivos de texto) o OutputStream (para archivos binarios)**.

- **Acceso aleatorio:** los ficheros de acceso aleatorio permiten **acceder** a sus datos **indicando una determinada posición desde la que leer/escribir**.

Ficheros binarios

- Contienen datos en un formato binario (en forma de bytes) no legible por humanos.
- Pueden almacenar cualquier tipo de información, como imágenes, sonidos, vídeos, archivos ejecutables, o cualquier estructura de datos compleja

Extensión	Tipo de fichero
.pdf	Fichero PDF
.jpg	Fichero de imagen
.doc, .docx	Fichero de Microsoft Word
.avi	Fichero de video
.ppt, .pptx	Fichero de PowerPoint

Escribir datos en un fichero binario

La escritura en **ficheros binarios** representa el **flujo de salida de bytes**, es decir, **necesitamos utilizar las clases que derivan de OutputStream**.

Se puede escribir datos en un fichero binario de muchas formas.

Veamos primero un ejemplo utilizando la clase **FileOutputStream** , y luego veremos un ejemplo utilizando además la clase **DataOutputStream.**

Escribir datos en un fichero binario

```
public static void main(String[] args) {
    String texto = "Hola";
    try {
        FileOutputStream fos = new FileOutputStream( name: "./src/ficheros/archivo.txt");
        // Convertimos el texto a bytes y los escribimos en el archivo
        fos.write(texto.getBytes());
        fos.write( b: 1);
        fos.close();
        System.out.println("Datos escritos en el archivo binario.");
    } catch (IOException e) {
        e.printStackTrace();
    }
```

// Se crea un nuevo fichero y se abre el fichero para escritura

// Se escribe información en el fichero.

// Siempre se tiene que cerrar el fichero.

Escribir datos en un fichero binario

```
public static void main(String[] args) {
    String texto = "Hola";
    try {
        DataOutputStream fos = new DataOutputStream(
                new FileOutputStream( name: "./src/ficheros/archivo_DOS.dat"));
        fos.writeInt( v: 1);          //Un entero ocupa 4 bytes y se
        fos.writeChar( v: 'a');        // escribirán en el archivo 4 bytes
        fos.writeBytes(texto);    // Se escribe cadena de texto
        fos.close();    // Siempre se tiene que cerrar el fichero.
    } catch (FileNotFoundException e) { //todas pueden derivar de IOException
        System.out.println(e.getMessage());
    } catch (IOException e) {
        System.out.println(e.getMessage());
    }
}
```

// Se crea un nuevo fichero y se abre el fichero para escritura

Escribir datos en un fichero binario

En los ejemplos anteriores; si ya existe el fichero en el que vamos a escribir, se eliminará y se creará uno nuevo.

FileOutputStream:

- solo acepta **bytes**
- El programador debe convertir datos a bytes
- Utiliza el método write()

DataOutputStream:

- Acepta **datos primitivos** como enteros, dobles, booleanos, etc
- Convierte automáticamente los datos primitivos a binario
- Puede utilizar writeInt(), writeDouble(), writeBoolean(), etc. para escribir datos primitivos.

Escritura en un fichero binario usando el búfer

Vamos a escribir más enteros en el fichero, pero utilizando **BufferedOutputStream**.

Se utiliza un búfer(bloque de memoria) para **almacenar temporalmente los bytes antes de escribirlos en disco**

Así se **reduce la cantidad de operaciones de escritura física (E/S) que se realizan en el sistema** y se **mejora el rendimiento cuando se realizan muchas escrituras pequeñas.**

El almacenamiento en búfer hace que las **operaciones de E/S sean más eficientes**. Para un programa que realiza E/S masivas, el almacenamiento en búfer es esencial.

La E/S es muy lenta en comparación con las operaciones con almacenamiento principal. **Sin almacenamiento en búfer, la E/S sería muy, muy lenta**.

Comparación utilizando ambos flujos para escribir muchos datos - DataOutputStream

```
public static void main(String[] args) {
    int cantidadDatos = 1000000;  // Un millón de enteros

    // Prueba con DataOutputStream (sin buffer)
    long inicioSinBuffer = System.currentTimeMillis();
    try {
        DataOutputStream dos = new DataOutputStream
                (new FileOutputStream( name: "sin_buffer.bin"));
        for (int i = 0; i < cantidadDatos; i++) {
            dos.writeInt(i);  // Escribir enteros en el archivo
        }
        dos.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
    long finSinBuffer = System.currentTimeMillis();
    System.out.println("Tiempo con DataOutputStream: " + (finSinBuffer - inicioSinBuffer)
}
```

```
C:\Users\cpgra\.jdks\openjdk-22.0.2\bin\java.exe
Tiempo con DataOutputStream: 28481 ms
```

Comparación utilizando ambos flujos para escribir muchos datos - BufferedOutputStream

```java
public static void main(String[] args) {
    int cantidadDatos = 1000000;  // Un millón de enteros

    // Prueba con BufferedOutputStream + DataOutputStream (con buffer)
    long inicioConBuffer = System.currentTimeMillis();
    try {
        DataOutputStream dosBuff = new DataOutputStream(
                new BufferedOutputStream(
                        new FileOutputStream( name: "con_buffer.bin")));
        for (int i = 0; i < cantidadDatos; i++) {
            dosBuff.writeInt(i);  // Escribir enteros en el archivo
        }
        dosBuff.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
    long finConBuffer = System.currentTimeMillis();
    System.out.println("Tiempo con BufferedOutputStream  " + (finConBuffer - inicioConBuffer)
}
```

```
C:\Users\cpgra\.jdks\openjdk-22.0.2\bin\java.exe
Tiempo con BufferedOutputStream  248 ms
```

Lectura de datos en un fichero binario

Para **leer un fichero, necesitaremos saber qué tipos de datos se usaron para escribir**, y así poder usar una subclase de InputStream que los pueda leer correctamente.

FileInputStream es una clase que permite leer bytes directamente desde un archivo.

Es la clase más básica para trabajar con archivos en formato binario en Java.

Se utiliza comúnmente para leer datos binarios, como imágenes, archivos de audio o archivos de vídeo, donde los datos no están organizados como texto, sino como bytes crudos.

Leer datos de un fichero binario con FileInputStream

```java
try {
    FileInputStream fis = new FileInputStream( name: "./src/ficheros/archivo_FOS.dat");
    //Se lee byte a byte
    int byteleido = fis.read();
    System.out.println(" byteleido = "+byteleido+", pasado a char = "+(char) byteleido);
    //Si queremos leer todos los bytes hasta llegar al final del fichero
    while ((byteleido = fis.read()) != -1) {  // Leer byte por byte
        System.out.print((char) byteleido);   // Convertir el byte leído a char y mostrarlo
    }
    fis.close();
} catch (FileNotFoundException e) { //todas pueden derivar de IOException
    System.out.println(e.getMessage());
} catch (IOException e) {
    System.out.println(e.getMessage());
}
```

DataInputStream

Características:

- Proporciona **métodos específicos** para leer datos primitivos como readInt(), readDouble(), readBoolean(), etc.
- Facilita la lectura de datos que se escribieron de forma estructurada en un formato binario utilizando DataOutputStream.
- Es **importante que los datos se lean en el mismo orden en que se escribieron**, ya que de lo contrario se producirá una interpretación incorrecta de los datos.

Leer datos de un fichero binario con DataInputStream

```
try {
    DataInputStream dis = new DataInputStream(
            new FileInputStream( name: "./src/ficheros/archivo_DOS.dat"));
    int intleido = dis.readInt();
    System.out.println(" intLeido = "+intleido);
    char charLeido = dis.readChar();
    System.out.println(" charLeido = "+charLeido);
    int byteleido;
    while((byteleido = dis.read())!=-1){
        System.out.print((char) byteleido); //ascii a char
    }
    dis.close();
} catch (FileNotFoundException e) { //todas pueden derivar de IOException
    System.out.println("FileNotFoundException: "+e.getMessage());
} catch (IOException e) {
    System.out.println("IOException: "+e.getMessage());
```

Leer datos de un fichero binario con BufferedInputStream.

Al igual que en la escritura para la lectura también disponemos de la clase **BufferedInputStream**.

La clase BufferedInputStream mejora el rendimiento al usar un buffer para almacenar temporalmente los datos leídos, lo que reduce la cantidad de operaciones de entrada/salida al disco.

Ejercicio
Demostrar que BufferedInputStream es más rápido que DataInputStream al realizar operaciones de E/S.
Para ello podéis utilizar los ficheros que se generaron para la comparación entre DataOutputStream y BufferenOutputStream.

Ficheros de caracteres

Las clases Java **FileWriter y FileReader** se utilizan para escribir y leer datos de archivos de texto (son clases de flujo de caracteres, **character stream**).

Se recomienda no utilizar las clases FileInputStream y FileOutputStream si vamos a leer o escribir información de texto.

Extensión	Tipo de fichero
.txt	Fichero de texto plano
.xml	Fichero XML
.json	Fichero de intercambio de información
.props	Fichero de propiedades
.conf	Fichero de configuración
.sql	Script SQL
.srt	Fichero de subtítulo

Escritura de caracteres

La clase Java **FileWriter** del paquete java.io se utiliza para escribir datos en forma de caracteres en un archivo.

- Esta clase hereda de la clase **OutputStreamWriter**.
- **FileWriter** se utiliza para abrir o crear un fichero y escribir en él carácter por carácter.
- **BufferedWriter** se usa para escribir texto de manera más eficiente, utilizando un buffer interno y permitiendo escribir líneas completas de texto.

Escritura de caracteres

Ejemplo:

```
public static void main(String[] args) {
    String str = "Este es un ejemplo de escritura usando FileWriter.";

    try {
        FileWriter fw = new FileWriter( fileName: "./src/ficheros/output.txt");
        fw.write(str);
        fw.close();
    } catch(IOException e) {
        System.out.println("Error E/S: " + e);
    }
}
```

Cuando se cierra el stream: close(), se escribe en el fichero.

Si quisiéramos grabar en el fichero en algún momento antes de cerrar, podemos usar el método **flush().**

Lectura de datos

FileReader es una clase en el paquete java.io que se usa para leer una secuencia de caracteres de los ficheros.

- Esta clase se hereda de la clase **InputStreamReader**.
- **FileReader** se utiliza para abrir el fichero y leerlo carácter por carácter.
- **BufferedReader** se usa para leer el texto de manera más eficiente, línea por línea, utilizando un buffer interno, **accediendo a los datos más rápidamente**.

Lectura de datos

Ejemplo:

```
try {
    FileReader fr = new FileReader( fileName: "./src/ficheros/output.txt");
    ch = fr.read();
    while (ch != -1) { //fin de fichero
        System.out.print((char) ch);
        ch = fr.read();
    }
    fr.close();
} catch (IOException fe) {
    System.out.println("Error de E/S");
}
```

Escritura y lectura con BufferedWriter y BufferedReader

```
try (BufferedWriter writer = new BufferedWriter(
        new FileWriter( fileName: "./src/ficheros/caracteres_Buff.txt"))) {
    writer.write( str: "Hola, mundo!");
} catch (IOException e) {
    e.printStackTrace();
}
try (BufferedReader reader = new BufferedReader(
        new FileReader( fileName: "./src/ficheros/caracteres_Buff.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```